Laboratory work 03

Objective – Learn how to use Terraform to provision and manage infrastructure in Microsoft Azure by deploying a Virtual Machine (VM).

Software – portal.azure.com; github.com, git, draw.io, cloudflare.com, terraform

In this lab, you will:

- Configure Azure authentication for Terraform.
- Write Terraform scripts to provision the following resources:
  - Resource Group
  - Virtual Network and Subnet
  - Network Security Group (NSG)
  - Public IP Address
  - Network Interface
  - Virtual Machine with SSH Access
- Deploy the infrastructure on Azure.
- Verify the deployment by connecting to the VM.
- Clean up resources to avoid unnecessary charges.

Tasks

1. **Install terraform - Install the latest version of Terraform on your local machine.**
2. **Install Azure CLI– Install Azure CLI for authentication.**

3. **Authenticate Azure Account**

Open the terminal and authenticate using the Azure CLI:

az login

4. **Create a Directory for the Lab**

For example:

Command
mkdir terraform-azure-vm
cd terraform-azure-vm

5. **Initialize Terraform**

Create a file named *main.tf* and define the provider:

```
provider "azurerm" {
  features {}
```

```
  subscription_id = "your subscribtion id"
}
```

Initialize the directory::

```
terraform init
```

6. **Write Terraform Configuration**

Add the following blocks to *main.tf*:

Resource Group

```
resource "azurerm_resource_group" "rg" {
  name = "terraform-lab-rg"
  location = "East US"
}
```

Virtual Network and Subnet

```
resource "azurerm_virtual_network" "vnet" {
  name = "terraform-lab-vnet"
  location = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  address_space = ["10.0.0.0/16"]
}


resource "azurerm_subnet" "subnet" {
  name = "terraform-lab-subnet"
  resource_group_name =   azurerm_resource_group.rg.name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes = ["10.0.1.0/24"]
}
```

Network Security Group

```
resource "azurerm_resource_group" "rg" {
  name = "terraform-lab-rg"
  location = "East US"
}
resource "azurerm_network_security_group" "nsg" {
  name = "terraform-lab-nsg"
  location = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name

  security_rule {
    name = "SSH"
    priority = 1001
    direction = "Inbound"
    access = "Allow"
    protocol = "Tcp"
    source_port_range = "*"
```

```
    destination_port_range = "22"
    source_address_prefix = "*"
    destination_address_prefix = "*"
  }
}
```

Public IP Address and Network Interface

```
resource "azurerm_public_ip" "public_ip" {
  name = "terraform-lab-pip"
  location = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  allocation_method = "Static"
}

resource "azurerm_network_interface" "nic" {
  name = "terraform-lab-nic"
  location = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name

  ip_configuration {
    name = "internal"
    subnet_id = azurerm_subnet.subnet.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id = azurerm_public_ip.public_ip.id
  }
}


resource "azurerm_linux_virtual_machine" "vm" {
  name                = "terraform-lab-vm"
  location            = azurerm_resource_group.rg.location
  resource_group_name = azurerm_resource_group.rg.name
  size                = "Standard_B1s"
  disable_password_authentication = false
  admin_username      = "azureuser"
  admin_password = "P@ssw0rd123!"
  network_interface_ids = [
    azurerm_network_interface.nic.id,
  ]

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "UbuntuServer"
    sku       = "18.04-LTS"
    version   = "latest"
  }
}
```

7. **Deploy the Infrastructure**

   Initialize the directory:

   ```
   terraform init
   ```

   Review the plan:

   ```
   terraform plan
   ```

   Apply the configuration:

   ```
   terraform apply
   ```

8. **Access the Virtual Machine via ssh**

9. **Alter code to assosiate NSG with the NIC**
   Deploy infrastructure
   Access the Virtual Machine via ssh

10. **Clean Up**
    Destroy the infrastructure to avoid unnecessary charges:

    ```
    terraform destroy
    ```

11. **Bonus Challenges (Optional):**

1. **Add Tags** – Add tags to all Azure resources for better management.
2. **Output Values** – Output the public IP of the VM in the Terraform configuration.
3. **SSH key** - Change ssh username/password authorisation type to the ssh key option

In your report, please provide:
- Work progress
- A copy of your *main.tf* configuration file.
- Show the resources belonging to the *terraform-lab-rg* resource group

Questions:
- What is Infrastructure as Code (IaC), and why is it important in modern cloud computing?
- What are the key benefits of using IaC tools like Terraform compared to manual cloud provisioning?
- Why did we create a Resource Group first in the deployment process?
- Why do we use *terraform init* and *terraform plan* before *terraform apply*?